FSMM: An Efficient Matrix Multiplication Accelerator Supporting Flexible Sparsity

ABSTRACT

Sparse matrix multiplication is a critical operation in deep learning. However, matrix sparsity leads to irregular data flow, which would degrade the efficiency of matrix multiplication. Traditional accelerators, equipped with additional hardware units to address this issue, often experience the issue of low hardware utilization. Furthermore, $N : M$ structured sparsity and corresponding hardware architectures face challenges such as accuracy degradation, limited flexibility, and restricted applicability. In this paper, we propose a Flexible Sparse Matrix Multiplication Accelerator (FSMM), which can improve the efficiency of sparse matrix multiplication through both algorithmic-level and hardware-level optimizations. At the algorithmic level, We propose the matrix-matrix multiplication with block-level outer production and fine-grained matrix reordering algorithm. The algorithm balances the sparsity of each column of a matrix block, which improves matrix compression, balances the load, and speeds up computation. This algorithm reduces storage by 8.2% $\sim$ 85.9%. At the hardware-level, we introduce a flexible architecture for matrix multiplication. It selects the most suitable data path to complete matrix multiplication based on the sparsity of the reordered matrix. FSMM achieves a speedup of 1.90× $\sim$ 16.18× over Systolic Array and 1.70× $\sim$ 2.87× over the existing TSTC approach.

CCS CONCEPTS

• **Hardware** → *Design modules and hierarchy*; **Hardware accelerators**; • **Computing methodologies** → Parallel algorithms; • **General and reference** → *Performance*.

KEYWORDS

Sparse Matrix-Sparse Matrix Multiplication, Deep learning, Matrix reordering

ACM Reference Format:

. xxxx. FSMM: An Efficient Matrix Multiplication Accelerator Supporting Flexible Sparsity. In *Proceedings of 2024 ACM/IEEE International Conference on Computer-Aided Design (ICCAD '24).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Artificial intelligence (AI) has been developing rapidly in recent years [23]. The proposal of various deep learning models has led to significant breakthroughs in deep learning in computer vision [11, 28, 29], audio and video processing [20, 25], natural language processing [2, 11, 12], and other fields. Matrix multiplication is the most computationally intensive and time consuming part of deep learning models [9, 26]. The resource consumption of matrix multiplication increases further with the increase of deep model size and the number of parameters. Using sparse matrices in the models is an efficient way to speed up the matrix multiplication [8].

Sparsity widespreads in deep learning models. In Convolutional Neural Network (CNN), sparsity of feature map is introduced through ReLU activation [3, 4] and weight sparsity is introduced through weight quantization and pruning [6, 7]. Matrix multiplication is thus converted to sparse-dense matrix multiplication or even sparse-sparse matrix multiplication. Sparse matrices introduced by ReLU activation or unstructured pruning have randomness, and this randomness makes the matrices have unpredictable data flow during computation. Some existing accelerators such as SIGMA [26], which selects appropriate data for computation through complex interconnection pathways, and OuterSPACE [22], which uses an additional result buffer on the outer product algorithm to implement sparse matrix multiplication, face the problems of data scheduling difficulties and poor hardware utilization.

$N : M$ structured sparse is another feasible solution. It allows the matrix to have the feature of N non-zero values in every M values by weight pruning to obtain a regular data flow [18, 24, 32]. However, this technique lacks flexibility and imposes large restrictions on the sparsity of the matrix which further reducing the accuracy of the results. For example, NVIDIA's tensor core [21] uses a 2:4 structured sparsity to obtain a 2× throughput. However, if the sparsity of the matrix is much lower than 50%, the speedup of this approach will be much lower than the expected value. Flightllm [31] and TSTC [14] use a fine-grained $N : M$ sparsity to flexibly handle matrix multiplication at various sparsity levels. But these approaches bring additional accuracy loss by further structured pruning on top of unstructured pruning. They also lack effective handling of the cases where both feature inputs and weight matrices are sparse.

To solve the above problems, we propose a matrix multiplication accelerator (**FSMM**) to efficiently deal with flexible sparsity. FSMM uses a fine-grained matrix reordering algorithm to align the sparsity in sub-blocks, which leads to a serials of different balanced $N : M$ structured sparsity in different sub-blocks. The reordering algorithm can balance the computational load, improve the computational efficiency of the hardware, and avoid Structured pruning that leads to accuracy degradation.

At hardware level, FSMM provides fully pipelined multiple datapaths to support different $N : M$ structured sparsity levels, which reuses the multiplier array and adder trees. A de-reordering unit is designed to recover the results of reordered matrices.

Our main contributions are listed as follows:

- **Fine-grained Matrix Reordering Algorithm.** we propose a Fine-grained Matrix Reordering algorithm (*FMR*) based

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCAD '24, October 27–31, 2024, NEW JERSEY, USA

© xxxx Copyright held by the owner/author(s). Publication rights licensed to ACM.

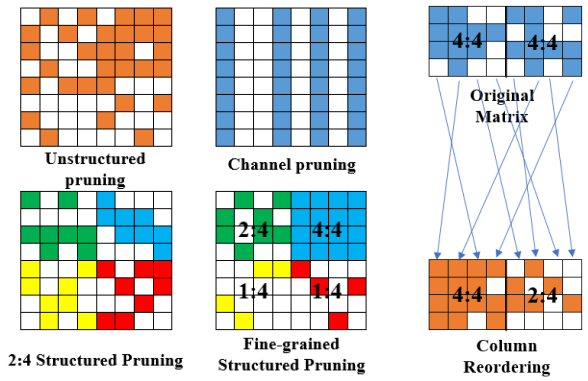
ACM ISBN xxx-x-xxxx-XXXX-X/xx/xx
https://doi.org/XXXXXXX.XXXXXXX

on Block-level Outer Production to reduce matrix storage space and increasing hardware utilization. We use *FMR* to adjust the data distribution of matrix to obtain sub-blocks with similar $N : M$ row/column sparsity, which improves the computational efficiency. We use a hybrid sparse matrix compression algorithm (*HSC*) to compress the sparse matrix block to reduce the storage space occupied by the matrix. These algorithms reduce the matrix space occupation by 8.2% ∼ 85.9%

- **Flexible hardware architecture supporting reordered sparse matrices.** We propose a flexible multi-sparsity matrix multiplication architecture to perform matrix multiplication with different sparsity levels. The architecture provides several fully pipelined datapaths for different sparsity levels. It selects the most appropriate data path for matrix multiplication based on the sparsity levels of the two input matrices. The architecture also contains a de-reordering unit to quickly recover the reordering matrix at the hardware level. Furthermore, for sparse-sparse matrix multiplication, we propose a strategy to adaptively select the sparser matrix as the stationary matrix to improve the efficiency.

We evaluated the performance of FSMM on ResNet50 [19]. Under equivalent hardware conditions, FSMM achieves a speedup of 1.90× ∼ 16.18× over Systolic Array and a speedup of 1.70× ∼ 2.87× over TSTC [14] in sparse-dense matrix multiplication case. In sparse-sparse matrix multiplication, FSMM obtains extra 10.4% improvement over the sparse-dense matrix multiplication case.

The rest of the paper is organized as follows. We present the background of sparse matrix multiplication and related works in Section 2. Then, Fine-grained Matrix Reordering Algorithm and Hardware Architecture of FSMM are presented in Section 3 and Section 4. Finally, the experimental results are presented in Section 5.



(a) Weight Pruning (b) Channel Reordering

Figure 1: Comparison of weight pruning and channel reordering. (a)Weighted pruning can obtain unstructured sparse, structured sparse and fine-grained structured sparse matrices, with additional accuracy loss. (b)Matrix column reordering can obtain sparse matrix sub-blocks with similar $N : M$ structured row sparsity without accuracy loss.

2 BACKGROUND AND RELATED WORKS

2.1 Weight Pruning

Weight pruning reduces the complexity and computational requirements by reducing the number of parameters in the model [16]. Weight pruning can be classified into two main classes: unstructured pruning [1] and structured pruning [15]. Unstructured pruning is flexible and can achieve higher compression rate but cannot directly accelerate the computation. Theoretically, by accurately selecting weights of low importance for pruning, unstructured pruning is able to achieve higher compression rate without significantly sacrificing model accuracy. However, the pruned model retains a random sparse level. Special hardware or software support is required to exploit the sparsity efficiently. Structured pruning can directly accelerate the computation but has the problems of limited compression rate and performance loss. Due to the need to maintain a specific structure, structured pruning may not be able to achieve as high compression rate as unstructured pruning. Figure 1(a) illustrates several types of weight pruning methods. Unstructured pruning allows the removal of weights at arbitrary locations. Channel pruning is used to reduce the computational complexity and runtime memory requirements of the model by removing some unimportant or redundant convolutional layer channels [13]. $N : M$ structured pruning requires N non-zero values for every M values [5]. Channel pruning and $N : M$ structured pruning can be computationally efficient, but lack the flexibility to achieve efficient compression. Fine-grained structured pruning [14] further prunes the model on top of unstructured pruning to maintain a $N : M$ structured sparsity at sub-block level, which ensures a certain degree of flexibility but introduces additional loss of accuracy.

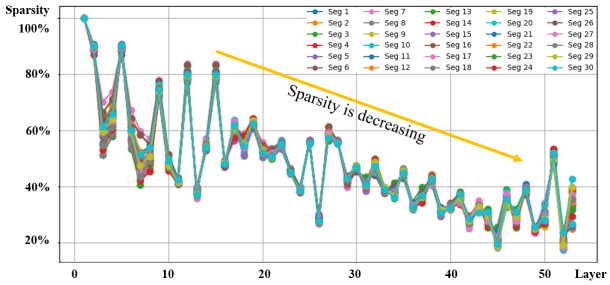


Figure 2: Variation of sparsity of convolutional input feature maps with the number of layers for different input images in ResNet50.

2.2 Flexible Structured Sparse Matrix Multiplication

Sparse Matrix multiplication is a very important computational operation in the field of deep learning [17]. Taking CNN as an example, im2col [30] is usually used in the convolutional layer of the network to convert the convolution to matrix multiplication. As shown in Figure 1(a), the convolution kernel (weight matrix) is sparse after pruning. As shown in Figure 2, due to the presence of activation layers such as ReLU, the feature maps of each convolutional layer become sparser as the number of convolutional layers increases.

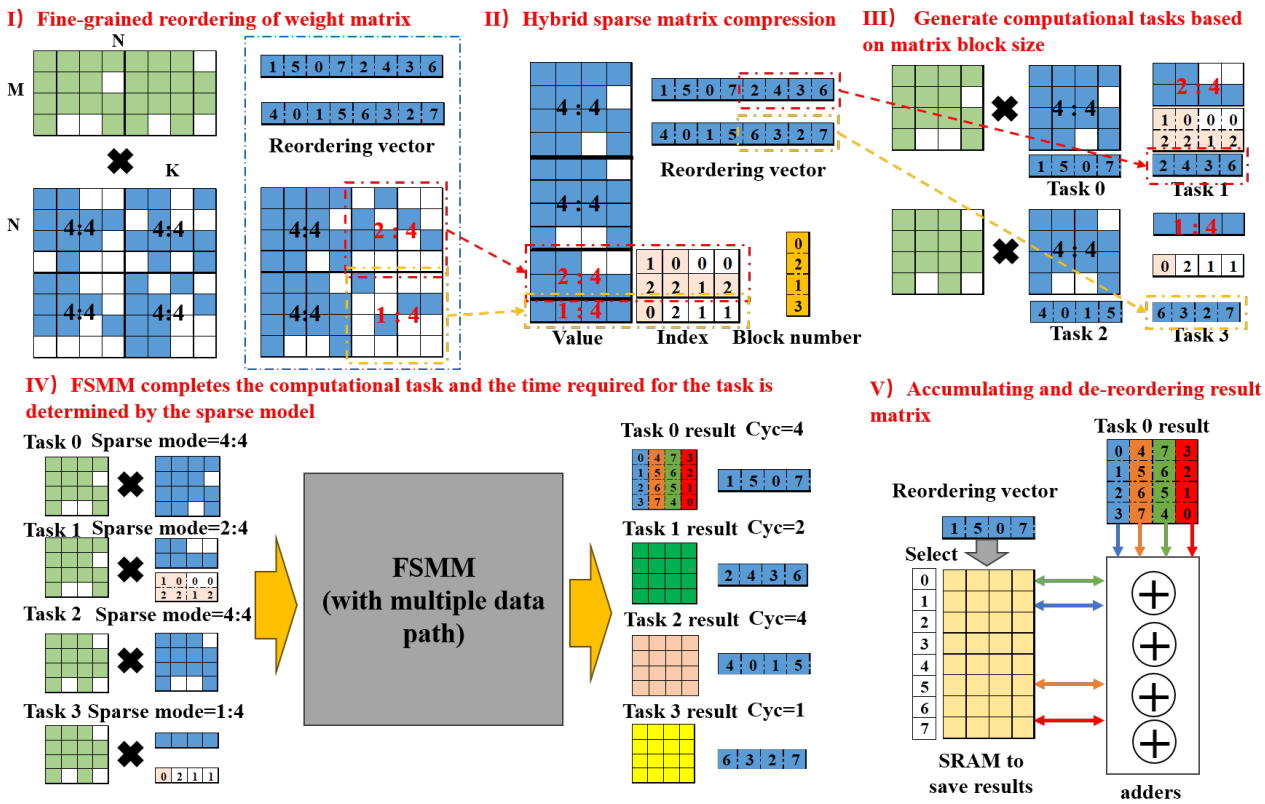
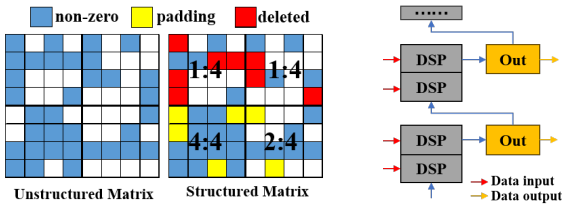


Figure 3: Workflow of FSMM from matrix reordering to result output. In this example, the size of the matrix block is set to 4×4.

The sparsity of the data, can drastically reduce the amount of effective computation. On the other hand, it leads to an irregular data flow. This irregular data flow is not hardware-friendly. As shown



(a) TSTC (b) Flightllm

Figure 4: Treatment of TSTC and Flightllm. (a)TSTC constructs structured sparse matrix blocks based on unstructured sparse matrices by padding and deletion. (b)Flightllm designs configurable sparse DSP chains.

in Figure 4(b), Flightllm [31] designs reconfigurable DSP chains. It divides several DSPs into one group and cascades each DSP group. The input data controls which group of DSP's output is the final result. This reconfigurable DSP chain allows Flightllm to support $N : M$ structured sparse matrix multiplication. However, not all matrices in deep learning models are structured sparse, and Flightllm cannot efficiently handle unstructured sparse matrix multiplication. TSTC [14] is a two-level sparse tensor core. As shown in Figure 4(a), based on the sparse level of the block, TSTC removes less influential elements from each matrix block if the number of elements in each row/column exceeds the sparse level limit. If there are not enough elements in each row (column), padding is used to fill them. It designs Dynamic Extendible Reduction Networks (DERN), which add output ports at each level of a addition tree to support multiple $N : M$ structure sparse matrix multiplications. By performing structured pruning on top of unstructured pruning, TSTC can process structured sparse matrices. However, the structured matrices have differences compared to the original matrices, so the method causes a loss of accuracy. And as the degree of structured pruning increases, the accuracy loss may further increase.

3 SPARSE MATRIX-MATRIX MULTIPLICATION WITH REORDERING

In this section, we will firstly introduce the sparse matrix-matrix multiplication using the block-level outer production algorithm. Then, we will introduce the reordering technique to align the sparsity in sub-blocks.

3.1 Matrix-Matrix Multiplication with Block-level Outer Production

We express the matrix-matrix multiplication in deep learning models as

$$O = F \times W, \tag{1}$$

where $F \in R^{m \times l}$ represents the feature map matrix, $W \in R^{l \times n}$ represents the weight matrix, $O \in R^{m \times n}$ represent the output matrix. Typically, the sparse pattern of W is known, while the sparse pattern of F is varying. In most of the hardware systems, only the sparse pattern of the weight matrix W is considered to accelerate the matrix-matrix multiplication.

Although the size of F or W would be extremely large, the size of the primitive of matrix-matrix multiplication in a hardware system is limited. We assume that the primitive of matrix-matrix multiplication in a hardware system is matrix multiplication of two matrices with size $q \times q$. For simplicity, we assume that m, n, l are divisible by q, i.e., $m = \hat{m} \times q$, $n = \hat{n} \times q$, $l = \hat{l} \times q$. If m, n, l are not divisible by q, the matrices F and W can be padded to satisfy the divisible condition. F and W matrices can thus be divided into blocks:

$$\begin{aligned} F &= [F_1, F_2, \cdots, F_{\hat{l}}] \\ W &= [W_1^T, W_2^T, \cdots, W_{\hat{l}}^T]^T, \end{aligned} \tag{2}$$

where $F_i \in R^{m \times q}$, and $W_i \in R^{n \times q}$.

$O = F \times W$ can then be expressed as the outer product of sub-blocks of F and W as follows.

$$\begin{aligned} O &= [F_1, F_2, \cdots, F_{\hat{l}}] \times [W_1^T, W_2^T, \cdots, W_{\hat{l}}^T]^T \\ &= \textstyle\sum_1^{\hat{l}} F_i \times W_i^T. \end{aligned} \tag{3}$$

F_i and W_i^T can be further divided into $q \times q$ sub-blocks, and $F_i \times W_i^T$ can be implemented by the matrix multiplication primitive of size $q \times q$ using outer multiplication. Assume that F_i and W_i can be further divided into sub-blocks as follows.

$$\begin{aligned} F_i &= [F_{i1}^T, F_{i2}^T, \cdots, F_{i\hat{m}}^T]^T \\ W_i &= [W_{i1}, W_{i2}, \cdots, W_{i\hat{n}}], \end{aligned} \tag{4}$$

where $F_{ij} \in R^{q \times q}$ and $W_{ij} \in R^{q \times q}$. Thus, $F_i \times W_i^T$ can be implemented using outer production as

$$F_i \times W_i^T = \begin{bmatrix} F_{i1}W_{i1} & F_{i1}W_{i2} & \cdots & F_{i1}W_{i\hat{n}} \\ F_{i2}W_{i1} & F_{i2}W_{i2} & \cdots & F_{i2}W_{i\hat{n}} \\ \vdots & & F_{ij}W_{ik} & \vdots \\ F_{i\hat{m}}W_{i1} & F_{i\hat{m}}W_{i2} & \cdots & F_{i\hat{m}}W_{i\hat{n}} \end{bmatrix}. \tag{5}$$

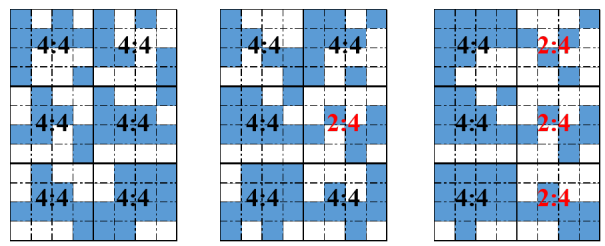
3.2 Balance Sparse Distribution via Reordering

In our hardware system, we set $q = 16$. A serial of datapaths supporting different sparse levels are implemented. In our implementation, 5 types of sparse levels, i.e., 1:16, 2:16, 4:16, 8:16 and 16:16 are implemented by reusing the multiplier array and the adder tree. These 5 types of structured sparsity levels are carefully selected to guarantee that the multiplier array can be fully reused. 8:16 means that for each column/row of the 16×16 matrix, there exist most 8 non-zeros in each column/row.

For the matrix multiplications in (5), we aim to adaptively select the most appropriate $N : M$ matrix multipliers datapath to complete the multiplications. For our implementation, the $N : M$ mode of 16×16 matrix block is determined by the maximum number of non-zeros over the 16 columns.

Take the original 4×4 matrix in Figure 1(b) as an example, where both matrix blocks contain a full column full of non-zeros. We can only consider them as sparse level of 4:4. Obviously this approach suffers from imbalance of non-zero distributions over columns. We need to balance distributions of non-zeros over columns for the matrix blocks.

Matrix reordering is an effective way to balance the sparsity of matrix columns. As shown in Figure 1(b), by reordering the columns of the matrix, we can convert two 4:4 matrix blocks into one 4:4 matrix block and one 2:4 matrix block. Each column of the reordered matrix block has similar sparsity.



(a) Original Matrix (b) Column reordering (c) Fine-grained Reordering

Figure 5: Matrix Reordering to obtain sparse matrix blocks. (a)The original matrix without reordering. (b)Matrix after column reordering. (c)Matrix after fine-grained reordering.

However, matrices in deep learning are large and reordering the entire channel (rows/columns of a matrix) may not result in the best sparse blocks, as shown in Figure 5(b). Therefore, we propose a fine-grained matrix reordering algorithm based on matrix-matrix multiplication with block-level outer production, which balances the sparsity of each column/row of a matrix block as much as possible, thus improving the computational efficiency, as shown in Figure 5(c).

We consider to reorder the fine-grained sub-matrices W_i one by one. For each W_i in (3), the columns of the matrix block are reordered in descending order according to the number of non-zeros in each column. After sorting, we get the reordered matrix W_i^* and the corresponding reordering order E_i, i.e.,

$$W_i^* = W_i \times E_i, \tag{6}$$

After reordering, W_i^* can also be divided into $q \times q$ sub-blocks as follows.

$$W_i^* = [W_{i1}^*, W_{i2}^*, \cdots, W_{i\hat{n}}^*], \tag{7}$$

Since the columns of W_i^* are reordered according to their number of non-zeros. Therefore, the columns of W_{ij}^* have similar number of non-zeros, as shown in Figure 5(c).

With the reordering, (3) can be expressed as

$$\begin{aligned} O &= [F_1, F_2, \cdots, F_{\hat{l}}] \times [W_1^T, W_2^T, \cdots, W_{\hat{l}}^T]^T \\ &= \sum_1^{\hat{l}} F_i \times W_i^T. \\ &= \sum_1^{\hat{l}} F_i^* \times W_i^* \times (E_i)^{-1} \\ &= \sum_1^{\hat{l}} O_i^* \times (E_i)^{-1} \end{aligned} \tag{8}$$

Equation 8 shows the relationship between the reordering matrix O_i^* and the intermediate results matrix O_i after fine-grained row reordering of matrix W. We only need to reorder the columns of the intermediate result matrix O_i according to the reordering order E_i and sum O_i to get the result matrix O.

Algorithm 1 Fine-grained Matrix Reordering Algorithm

Input: Matrix W of size $n \times m$
Output: reordering matrix W_i^* and reordering vector $index_i s$

```
for each W_i in W do
    for each column ∈ W_i do
        Set nnzs_i[column] as the number of non-zeros of column.
    end for
    [index_i, sorted_nnzs] = sort(nnzs_i)
    W_i^* = W_i[:, index_i]
end for
```

Algorithm 1 shows pseudo-code for fine-grained matrix reordering algorithm. For the weight matrix W, we firstly divide it into sub-blocks as shown in (2). Then, for each column of W_i, the number of non-zero elements is counted and the matrix is reordered according to the number of non-zero elements. The reordered matrix W^* is the combination of the sub-matrices W_i^*. The algorithm also generates reordering vectors for recovering the final result matrix. After reordering the matrices, we make each column of matrix block have similar sparse patterns, thus improving the computational efficiency. For the reordered matrix, it can also be pruned to further reduce the model size and speed up the matrix multiplication.

Then, we need to compress the reordering matrix according to their sparse levels, thus reducing the storage space of the matrices and increasing the computational efficiency. We use a hybrid sparse matrix compression algorithm (HSC) to compress the reordering matrix. For each W_{ij}^* in (7), we count the maximum value of the number of non-zero per column in each matrix block and generate the sparse mode for that block. For each matrix block of size $q \times q$, we compress it into an $\left(a \times \frac{N^*}{16}\right) \times a$ value matrix and an index matrix of the same size according to the maximum number of non-zero elements N^* in each column of the matrix.

4 HARDWARE ARCHITECTURE

4.1 Overview

FSMM is a fully pipelined design, which can efficiently implement matrix multiplication computations of arbitrary size and arbitrary sparsity level, while guaranteeing computational efficiency. FSMM reuses most of the hardware units in different sparse levels and has a high hardware utilization rate. As shown in Figure 6, the architecture of FSMM consists of Matrix Buffer Unit, Control Unit, Data Spilt Unit, Computation Unit, and De-Reordering Unit. The weight matrix is loaded into the weight cache of the Matrix Buffer Unit via high bandwidth memory (HBM) after offline reordering by the fine-grained matrix reordering algorithm, and the feature matrix is loaded from the HBM into the feature cache of the Matrix Buffer Unit. The Control Unit controls the Matrix Buffer Unit to output the feature matrix block and weight matrix block to the data Spilt Unit based on the control commands. The Data Spilt Unit selects the appropriate data path based on the sparse level of the weight matrix block and the feature matrix block. The Computation Unit completes the matrix multiplication computation based on the sparse mode and the input data. The result of the computation is de-reordered in the De-Reordering Unit according to the reordering vector and the De-Reordering Unit outputs the result to the HBM.

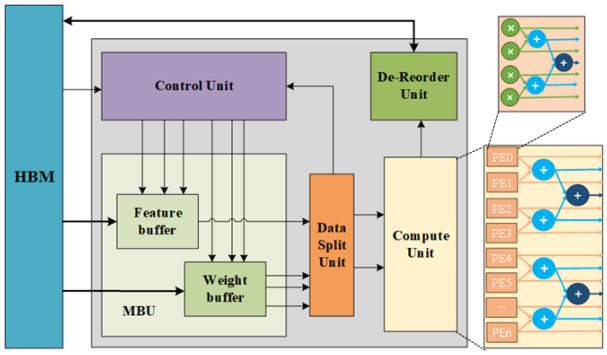
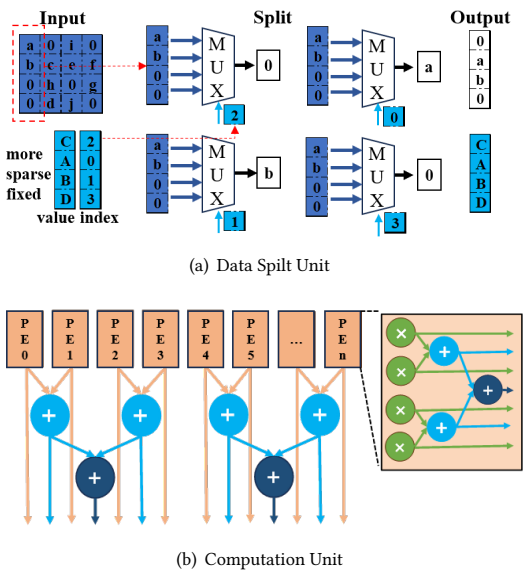


Figure 6: The overall architecture includes Matrix Buffer Unit, Control Unit, Data Spilt Unit,Computation Unit, and De-Reordering Unit.

4.2 Data Spilt Unit And Computation Unit

We design the Data Spilt Unit And Computation Unit in such a way that the accelerator can reuse most of the hardware units at different sparsity levels. Figure 7(a) shows the data flow of Data Spilt Unit when the matrix block size is 4×4. Data Spilt Unit, after receiving the matrix block information provided by the Matrix Buffer Unit, firstly compares the sparsity levels of the two matrix blocks, and the sparser matrix block will be selected as the stationary matrix. This operation makes our architecture well-adapted to the case of sparse-sparse matrix multiplication. The output corresponding to the stationary matrix will remain unchanged until the multiplication of the current matrix block is completed. The index matrix of the stationary matrix will be used as the pick-through address of the data selector to select the flowing matrix data for output. In different sparse levels, the Data Spilt Unit selects different data paths to provide enough data for the computation of the Computation Unit. For example, in 1:16 mode, the Data Spilt Unit selects all 16 rows in one cycle and outputs them, while in 16:16 mode, the Data Spilt Unit selects 1 row in one cycle and outputs them.

As shown in Figure 7(a), the Computation Unit is a fully pipelined two-stage structure, the Computation Unit contains 64 PEs (Processing Engine) and corresponding reconfigurable addition trees. Each PE contains 4 multipliers and a corresponding reconfigurable addition tree. At the PE level, the output of each multiplier and



(a) Data Spilt Unit

(b) Computation Unit

Figure 7: Hardware architecture of Data Spilt Unit and Computation Unit. (a)Dataflow of Data Spilt Unit when the matrix block size is 4×4. (b)Computation Unit contains 64 PEs. Each PE contains 4 multipliers.

each level of the addition tree can be used as the output of a PE. At the Computation Unit level, the output of each PE and each level of the addition tree can be used as the output of the Computation Unit. As a result, the computational unit can output the results at different stages depending on the sparse level of the input data, thus enabling the multiplier and addition tree to be efficiently reused.

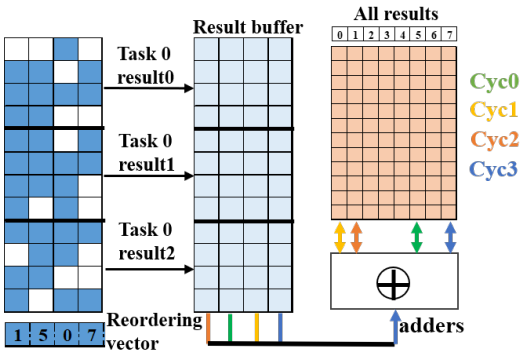


Figure 8: De-Reordering Unit contains an intermediate result buffer and an adder array consisting of 128 adders.

4.3 De-Reordering Unit

The De-Reordering Unit is to complete the matrix de-reordering and the accumulation of matrix blocks to get the final output matrix. As shown in Figure 8, the De-Reordering Unit contains an intermediate result buffer and an adder array consisting of 128 adders. When a number of matrix block multiplications have been completed, the data in the result buffer will be de-reordered according to the reordering vector corresponding to the matrix block. In the de-reordering process, the reordering unit first reads the matrix columns from the SRAM, based on the addresses corresponding to the reordering vector. The matrix columns and the corresponding columns in the result buffer are summed by the adder unit. The result of the addition operation is written back to the SRAM address corresponding to the reordering vector, thus completing the summation and reordering. In addition, adding a FIFO between the result buffer and the accumulation can improve the efficiency.

4.4 Summary of FSMM

The workflow of FSMM is demonstrated in Figure 3. Here we set the size of the matrix block to 4×4, and the size of the de-reordering unit buffer to one matrix block. We summarize the flow as follows:

1) The matrix is reordered and the reordering vector is generated by using FMR. Compared to the original matrix, we obtain two sparse matrix blocks and a more stable data flow for each matrix block.
2) The reordered matrix is compressed to get the matrix in compressed format by using HSC. The compressed matrix occupies less storage space and facilitates matrix computation.
3) The matrix multiplication is divided into a series of tasks and computational instructions are generated to control the hardware computation.
4) The hardware architecture adapts to the sparsity level of each task to complete the matrix multiplication of each matrix block, and the time required to complete each task is determined by the sparsity level.
5) The matrix columns are written to the addresses according to the corresponding reordering vectors of the task. After completing the accumulation of the matrix columns, the result is written back, thus completing the matrix de-reoedering and accumulation.

5 EXPERIMENTAL RESULTS

5.1 Experimental Setup

To evaluate the performance of our proposed FSMM architecture, we implement it on RTL. We set the matrix block size to 16×16. The Computation Unit consists of 64 PEs and reconfigurable addition trees, each PE consists of 4 multipliers and corresponding reconfigurable addition tree. The De-Reordering Unit consists of an adder array of 128 adders. In addition, we designed an emulator to simulate the hardware behavior and evaluate the runtime cycles. Based on our RTL, the area of our design is evaluated in Synopsys Design Compiler [27] using a 28nm library.

We compare FSMM with several state-of-the-art architectures. We completed the RTL design of NVIDIA's 2:4 structured sparse tensor core [21] and Google TPU's Systolic Array [10] while ensuring the same number of multipliers and peak throughput. TSTC is an advanced fine-grained structured sparse matrix multiplication accelerator. We completed its RTL design based on a matrix block size of 16×16. No further structured pruning is done on top of unstructured pruning in order not to incur accuracy loss.

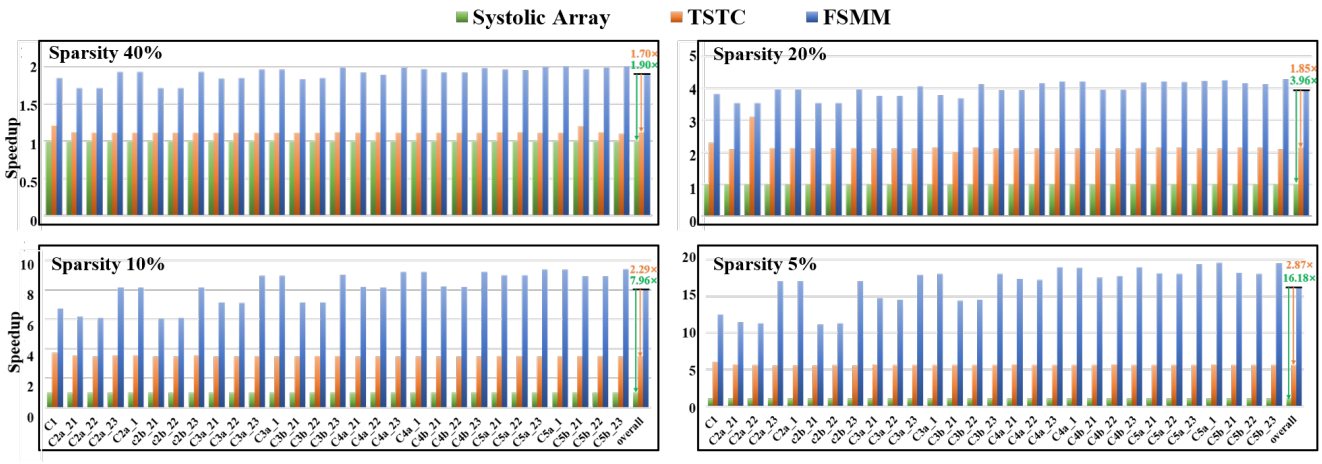


Figure 9: On the ResNet50 model, with weight sparsity in the range of 40% ∼ 5%, **and FSMM achieves a speedup of** 1.90× ∼ 16.18× **over Systolic Array and a speedup of** 1.70× ∼ 2.87× **over TSTC.**

We evaluate matrix multiplication using deep learning models. We randomly generated matrix multiplication datasets of typical size in the ResNet50 [19]. The dataset consists of matrices with different sparsities, the sparsity of each matrix is obtained by random unstructured pruning. Unstructured pruning means that we do not impose any restriction on weight pruning to reflect the wide applicability of FSMM. We also tested the performance of FSMM for the dense case and the $N : M$ structured sparse case. This treatment facilitates performance comparison with baseline.

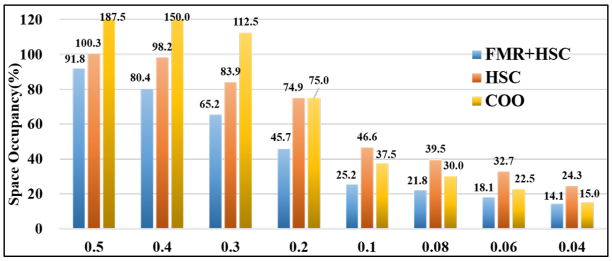


Figure 10: Matrix space usage compared to dense, HSC, and coo formats. With 100% space occupancy for the dense format, in the sparsity range of 50% ∼ 4% **the FMR+HSC method reduces the space occupancy from** 91.8% ∼ 14.1%**.**

5.2 Performance of FMR

We use different methods to store sparse matrices with different sparsity level (Percentage of non-zero values) to demonstrate the efficiency of our proposed FMR. Figure 10 demonstrates the advantages of our proposed FMR+HSC approach over the dense and COO formats for data storage. Firstly, in terms of effective data read/write capability, HSC occupies less space in the sparsity interval of 50% ∼ 20% while having the same or even better effective data read/write capability as COO format. This advantage effectively reduces the memory overhead of our architecture in preserving sparse weight matrix and speeds up the matrix multiplication.

This advantage is further extended with the introduction of FMR, and the FMR+HSC method has better effective data read/write capability than the COO format in the 50% ∼ 4% sparsity range while occupying only 91.8% ∼ 14.1% of the memory space for the dense case. In terms of data compression rate (1-Space Occupancy), in the sparsity range of 50% ∼ 4%, the FMR+HSC method improves the data compression rate from 8.2% ∼ 85.9% which is 1.01× ∼ 2.17× of the COO method, and 1.18× ∼ 10.89× of the HSC method alone. This effect is also reflected in the accelerated calculations.

5.3 Performance of Matrix Multiplication

We evaluate the performance of different accelerators in the ResNet50 [19]. We use a dense Systolic Array as the baselines. We compare our approach with TSTC [14], Systolic Array [10], and 2:4 Tensor-Core [21]. For a fair and comprehensive comparison, we compare sparse-dense matrix multiplication and sparse-sparse matrix multiplication using the same sparsity and size, respectively. In the case of dense matrix multiplication and structured sparse matrix multiplication, FSMM has the same computational speed as 2:4 tensor core[21] and TSTC [14]. As shown in Figure 9, we report the computational speedups of sparse matrix-dense matrix multiplication for FSMM, TSTC, and Systolic Array at different weight sparsities on a layer-by-layer basis and compute the average speedup for each architecture.

Overall, FSMM provides great speedup for sparse-dense matrix multiplication at different convolutional layers and sparsity levels. In the case of weight sparsity decreasing from 40% to 5%, FSMM achieves a speedup of 1.90× ∼ 16.18× over Systolic Array and a speedup of 1.70× ∼ 2.87× over TSTC. There are some differences in FSMM speedup for different convolutional layers. The different sparse patterns of the weight matrix and the different sizes of matrix multiplication for different convolutional layers are the main reasons for this difference. Specifically, the higher the number of output channels, the better the speedup. The speedup of FSMM varies for different sparsity levels. The effectiveness of FMR and HSC for matrices with different sparsity levels is the main reason

for this difference. As the matrix becomes sparse, FMR can produce more regular sparse matrices, and thus HSC can be compressed by a higher compression rate. Thus, the hardware utilization increases. Benefiting from our flexible architecture, FSMM supports

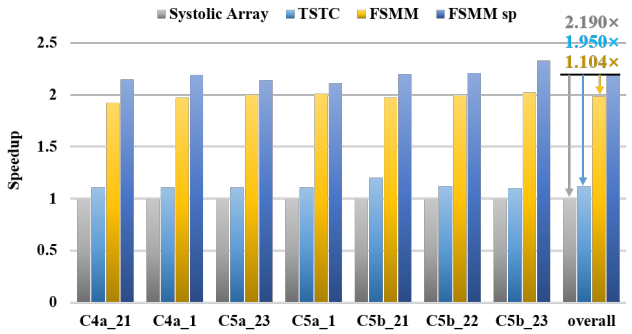


Figure 11: FSMM computational speedup for performing sparse matrix-sparse matrix multiplication in ResNet50. The weight sparsity is 40% and the convolutional layer feature input sparsity is 20% ∼ 30%**.**

sparse-sparse matrix multiplication. We prune the weight matrix to 40% sparsity to test the speedup. Figure 11 illustrates the acceleration effect in the case of sparse-sparse matrix multiplication. The computational speed of FSMM is further improved by 10.4% in the sparse-sparse matrix multiplication case compared to the sparse-dense matrix multiplication case. Meanwhile, the computational speed of FSMM, in this case, is 1.95× of TSTC [14] and 2.19× of Systolic Array [10]. As the sparsity of the feature input matrix is further increased, the computational speed of FSMM will be further increased.

5.4 Power and Area Analysis

We evaluated the area and power consumption of the logic cells and buffers of the FSMM using Synopsys Design Compiler [27] with a 28nm library.

Table 1: Power and area estimates for FSMM

Module	Area(mm^2)	Power(mW)
Matrix Buffer Unit	0.107	97.323
Control Unit	0.0001	0.009
Data Spilt Unit	0.044	17.934
Computation Unit	0.121	81.538
De-Reordering Unit	0.108	49.749
Total	0.380	246.55

The detailed hardware overhead results of the FSMM are shown in Table 1. The total area overhead of the FSMM is 0.380 mm^2 and the total power consumption is 246.55 mW. Computation Unit and Matrix Buffer Unit are the main sources of area and power consumption. Compared to TSTC [14], FSMM uses only 44% of the area and 86% of the power consumption.

6 CONCLUSION

In this work, we show that efficient matrix multiplications can be achieved by matrix reordering without further pruning of the deep learning models. We propose the matrix-matrix multiplication with block-level outer production and fine-grained matrix reordering algorithm that allow unstructured sparse matrices to be saved in a hardware-friendly form and fed into the accelerator to complete the matrix multiplication computation. It reduces the matrix space occupation by 8.2% ∼ 85.9%. Then, we design a dedicated matrix multiplication accelerator on the premise of the above algorithms to achieve efficient computation of matrix multiplication with arbitrary sparsity. In sparse-dense matrix multiplication, FSMM achieves a speedup of 1.90× ∼ 16.18× over Systolic Array and 1.70× ∼ 2.87× over TSTC. In sparse-sparse matrix multiplication, FSMM obtains extra 10.4% improvement over the sparse-dense matrix multiplication case.

REFERENCES

[1] AR Aswani, R Chithra, and Alex Pappachen James. 2022. Unstructured weight pruning in variability-aware memristive crossbar neural networks. In *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, IEEE, Austin, TX, USA, 3458–3462.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. 2020. Language Models are Few-Shot Learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[3] Shijie Cao, Lingxiao Ma, Wencong Xiao, Chen Zhang, Yunxin Liu, Lintao Zhang, et al. 2019. Seernet: Predicting convolutional neural network feature-map sparsity through low-bit quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Long Beach, CA, USA, 11216–11225.

[4] Pengcheng Dai, Jianlei Yang, Xucheng Ye, Xingzhou Cheng, Junyu Luo, Linghao Song, et al. 2020. SparseTrain: Exploiting dataflow sparsity for efficient convolutional neural networks training. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, IEEE, San Francisco, CA, USA, 1–6.

[5] Chao Fang, Wei Sun, Aojun Zhou, and Zhongfeng Wang. 2024. Efficient N:M Sparse DNN Training Using Algorithm, Architecture, and Dataflow Co-Design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 2 (2024), 506–519. https://doi.org/10.1109/TCAD.2023.3317789

[6] Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. 2023. Depgraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Vancouver, BC, Canada, 16091–16101.

[7] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. arXiv:1510.00149 [cs.CV]

[8] Guyue Huang, Guohao Dai, Yu Wang, and Huazhong Yang. 2020. Ge-spmm: General-purpose sparse matrix-matrix multiplication on gpus for graph neural networks. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, IEEE, Atlanta, GA, USA, 1–12.

[9] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. 2021. Data movement is all you need: A case study on optimizing transformers. *Proceedings of Machine Learning and Systems* 3 (2021), 711–732.

[10] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Bajwa, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, Toronto, ON, Canada, 1–12. https://doi.org/10.1145/3079856.3080246

[11] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. 2022. Transformers in vision: A survey. *ACM computing surveys (CSUR)* 54, 10s (2022), 1–41.

[12] Ngoc Tran Khanh Le, Nadia Hadiprodjo, Hazem El-Alfy, Aziz Kerimzhanov, and Avtandil Teshebaev. 2023. The Recent Large Language Models in NLP. In *2023 22nd International Symposium on Communications and Information Technologies (ISCIT)*. IEEE, Sydney, Australia, 1–6. https://doi.org/10.1109/ISCIT57293.2023.10376050

[13] Mingbao Lin, Rongrong Ji, Yuxin Zhang, Baochang Zhang, Yongjian Wu, and Yonghong Tian. 2020. Channel Pruning via Automatic Structure Search. arXiv:2001.08565 [cs.CV]

[14] Jun Liu, Guohao Dai, Hao Xia, Lidong Guo, Xiangsheng Shi, Jiaming Xu, et al. 2023. TSTC: Two-level Sparsity Tensor Core Enabling both Algorithm Flexibility and Hardware Efficiency. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, IEEE, San Francisco, CA, USA, 1–9.

[15] Siping Liu, Qiang Li, Bo Peng, Yanping Qiao, Maosong Lin, and Qiang Zeng. 2022. An Energy-Constrained Optimization-Based Structured Pruning Method for Deep Neural Network Compression. In *2022 IEEE 22nd International Conference on Communication Technology (ICCT)*. IEEE, IEEE, Nanjing, China, 1903–1907.

[16] Yiding Liu, Yinglei Teng, and Tao Niu. 2023. Splittable Pattern-Specific Weight Pruning for Deep Neural Networks. In *2023 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, IEEE, Brisbane, Australia, 1439–1444.

[17] Sharad Malik and Pareesa Ameneh Golnari. 2019. Sparse matrix to matrix multiplication: a representation and architecture for acceleration. In *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, Vol. 2160. IEEE, IEEE, New York, NY, USA, 67–70.

[18] Huizi Mao, Song Han, Jeff Pool, Wenshuo Li, Xingyu Liu, Yu Wang, and William J Dally. 2017. Exploring the granularity of sparsity in convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*. IEEE, Honolulu, HI, USA, 13–20.

[19] Sheldon Mascarenhas and Mukul Agarwal. 2021. A comparison between VGG16, VGG19 and ResNet50 architecture frameworks for Image Classification. In *2021 International conference on disruptive technologies for multi-disciplinary research and applications (CENTCON)*, Vol. 1. IEEE, IEEE, Bengaluru, India, 96–99.

[20] Daniel Michelsanti, Zheng-Hua Tan, Shi-Xiong Zhang, Yong Xu, Meng Yu, Dong Yu, and Jesper Jensen. 2021. An Overview of Deep-Learning-Based Audio-Visual Speech Enhancement and Separation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 29 (2021), 1368–1396. https://doi.org/10.1109/TASLP.2021.3066303

[21] NVIDIA. 2023. *Nvidia tensor cores*. NVIDIA. Retrieved April 14, 2024 from https://www.nvidia.com/en-us/data-center/tensor-cores/

[22] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, et al. 2018. Outerspace: An outer product based sparse matrix multiplication accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, IEEE, Vienna, 724–736.

[23] Yunhe Pan. 2016. Heading toward artificial intelligence 2.0. *Engineering* 2, 4 (2016), 409–413.

[24] Jeff Pool and Chong Yu. 2021. Channel permutations for n: m sparsity. *Advances in neural information processing systems* 34 (2021), 13316–13327.

[25] Hendrik Purwins, Bo Li, Tuomas Virtanen, Jan Schlüter, Shuo-Yiin Chang, and Tara Sainath. 2019. Deep learning for audio signal processing. *IEEE Journal of Selected Topics in Signal Processing* 13, 2 (2019), 206–219.

[26] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, et al. 2020. Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, IEEE, San Diego, CA, USA, 58–70.

[27] Synopsys. 2023. *Design compiler (synopsys.com)*. Synopsys. Retrieved April 14, 2024 from https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html

[28] Juan Terven, Diana-Margarita Córdova-Esparza, and Julio-Alejandro Romero-González. 2023. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction* 5, 4 (2023), 1680–1716. https://doi.org/10.3390/make5040083

[29] Athanasios Voulodimos, Nikolaos Doulamis, Anastasios Doulamis, Eftychios Protopapadakis, and Diego Andina. 2018. Deep Learning for Computer Vision: A Brief Review. *Intell. Neuroscience* 2018 (jan 2018), 13 pages. https://doi.org/10.1155/2018/7068349

[30] Haoyu Wang and Chengguang Ma. 2021. An optimization of im2col, an important method of cnns, based on continuous address access. In *2021 IEEE International Conference on Consumer Electronics and Computer Engineering (ICCECE)*. IEEE, IEEE, Guangzhou, China, 314–320.

[31] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, et al. 2024. FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays* (Monterey, CA, USA) *(FPGA '24)*. Association for Computing Machinery, New York, NY, USA, 223–234. https://doi.org/10.1145/3626202.3637562

[32] Aojun Zhou, Yukun Ma, Junnan Zhu, Jianbo Liu, Zhijie Zhang, Kun Yuan, et al. 2021. Learning N:M Fine-grained Structured Sparse Neural Networks From Scratch. arXiv:2102.04010 [cs.CV]

Received xx xxxx xxxx; revised xx xxxx xxxx; accepted xx xxxx xxxx